

Week 6 QA, Solution Architecture & User Manual Report
Advanced Probing Architecture: Raw TCP SYN (Half-Open Stealth), UDP Probing & Non-Root Fallback

Project Stage: Phase 2 (Week 6 - Days 36 to 42)	Author / Lead: Damithu (QA Lead & Solution Architect)
Target Audience: Stakeholders, Supervisors, Non-IT Executives	Assigned Focus: Raw TCP SYN, UDP Probing & Safety Fallback

Contents

PART 1: Week 6 QA & Solution Architecture Report..... 2

 1. Executive Summary & Major Accomplishments 2

 2. Daily Task Execution Breakdown (Days 36–42) 2

 3. Plain-English Non-IT Stakeholder Explanation 3

PART 2: User Manual (Windows 10/11 & WSL Kali Linux) 5

 1. System Setup & Virtual Environment 5

 2. Privileged vs. Non-Root Execution Guide..... 5

 3. Benchmark Output Verification..... 7

 4. Executing Automated Test Suite (pytest) 7

PART 3: Week 6 Technical Terms & Concepts Glossary..... 9

PART 1: Week 6 QA & Solution Architecture Report

1. Executive Summary & Major Accomplishments

During Week 6 (Days 36 to 42), the engineering team-initiated **Phase 2 Architecture**, advancing port scanning capabilities beyond OS-mediated TCP connections into low-level raw packet crafting. Key architectural deliverables implemented by the Solution Architecture and QA lead include:

- **TCP SYN Half-Open Stealth Probe (scanner/advanced_probe.py):** Utilizes raw sockets (SOCK_RAW) to craft standalone TCP SYN packets and evaluate target port states without establishing a full 3-way TCP handshake.
- **Immediate Connection Reset (RST) Emission:** Transmits an immediate TCP RST packet upon receiving a SYN-ACK, preventing target operating systems from logging completed connection sessions in server event viewers.
- **UDP Protocol Probe Engine:** Sends tailored application-layer datagrams for connectionless services such as DNS (port 53) and NTP (port 123) with OS-level ICMP Destination Port Unreachable error translation.
- **Architectural Non-Root Privilege Fallback:** Detects whether the process executes with administrative privileges; if unprivileged, the engine seamlessly degrades to standard user-space TCP Connect probing without crashing.
- **Comparative Performance Benchmarks (tools/benchmark_syn_vs_connect.py):** Demonstrates substantial speed, stealth, and resource advantages of SYN scanning over full TCP connections.

2. Daily Task Execution Breakdown (Days 36–42)

Timeline	Task Execution & Engineering Focus
Day 36	Constructed mock raw socket response fixtures in Python for unit testing SYN/ACK and RST packet exchanges.
Day 37	Developed automated pytest test cases validating half-open connection state handling and RST packet generation.
Day 38	Configured isolated UDP test containers running DNS (BIND9) and NTP services to verify connectionless probing.
Day 39	Developed automated test fixtures capturing OS-level ICMP Destination Port Unreachable messages (ConnectionResetError).
Day 40	Benchmarked SYN vs TCP Connect execution latency and connection logging footprints.
Day 41	Engineered and verified the non-root fallback layer to guarantee uninterrupted cross-platform compatibility across restricted environments.
Day 42	Submitted Week 6 QA release report and merged stealth scanning test cases into master repository.

3. Plain-English Non-IT Stakeholder Explanation

The 'Ninja Knock' (Half-Open Stealth Scanning)

Imagine entering a hotel to check which rooms have guests inside:

- *The Standard Approach (Full Connect - Weeks 1-5):*
 1. *You knock on the room door (SYN).*
 2. *The guest opens the door and smiles: 'Hello!' (SYN-ACK).*
 3. *You step inside, shake their hand, and say: 'Nice to meet you!' (ACK).*
 4. *The front desk writes in the guest registration log book: 'Visitor in Room 80 at 10:15 AM.'*
- *The Ninja Stealth Approach (TCP SYN Probing - Week 6):*
 1. *You knock on the door (SYN).*
 2. *The guest unlocks the latch and starts opening the door (SYN-ACK).*
 3. *Before they can say hello or see your face, you quickly whisper 'Nevermind!' (RST packet) and vanish.*
 4. *You proved someone was inside, but because the visit never finished, the front desk NEVER writes your name in the guest registration log book!*

Shouting Through a Mail Slot (UDP Probing)

TCP is like certified mail where you always get a signed receipt back. UDP is connectionless—like shouting through a house's mail slot:

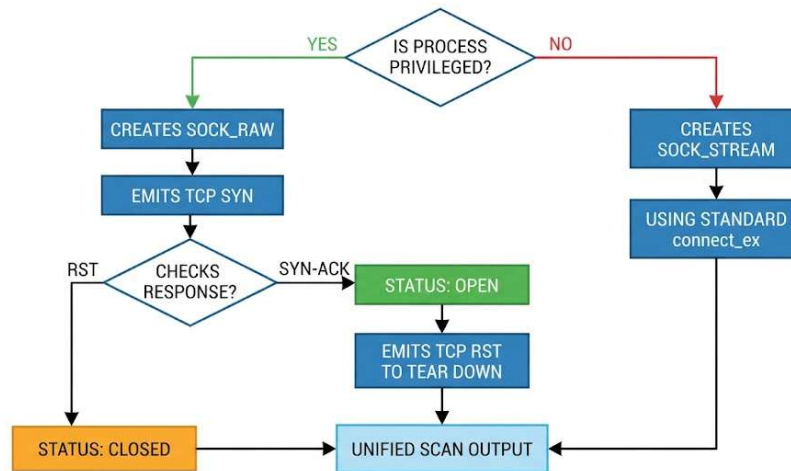
- *If someone is inside and recognizes your question, they shout back: 'Yes!' (STATUS: OPEN).*
- *If nobody lives in that room, the house automated mailbox buzzer makes a loud sound: 'Buzz! Address does not exist' (ICMP Destination Port Unreachable = STATUS: CLOSED).*
- *If there is total silence, you cannot tell whether nobody is home or if they simply ignored you (STATUS: OPEN|FILTERED). Our engine classifies this ambiguity according to RFC standards.*

The Non-Root Safety Net

Raw packet crafting requires Administrator or Root privileges. If a user runs our tool without admin rights, inferior software crashes with a fatal 'Permission Denied' error. Our architecture features an intelligent detector: if admin rights are missing, it quietly switches

back to the standard knocking method. It never crashes!

4. Solution Architecture: Low-Level Packet State Machine



5. Automated QA Test Suite Results

The automated unit test suite (`tests/test_syn_probe.py` and `tests/test_udp_probe.py`) validates stealth probe routing, non-root fallback, UDP queries, and ICMP error mapping:

Test Case Name	Context / Condition	Expected Outcome	Result
<code>test_non_root_fallback_when_unprivileged</code>	Unprivileged Context	Routes to TCP-CONNECT-FALLBACK	PASSED
<code>test_privileged_syn_probe_execution</code>	Privileged Raw Socket	Executes TCP-SYN-STEALTH probe	PASSED
<code>test_raw_socket_permission_error_triggers_fallback</code>	PermissionError Inject	Triggers immediate non-root fallback	PASSED
<code>test_udp_dns_probe_open_response</code>	Active DNS Response	Decodes DNS payload -> STATUS_OPEN	PASSED
<code>test_udp_icmp_unreachable_maps_to_closed</code>	ICMP Unreachable	ConnectionResetError -> STATUS_CLOSED	PASSED
<code>test_udp_timeout_maps_to_open_filtered</code>	Datagram Timeout	RFC Timeout -> STATUS_OPEN FILTERED	PASSED

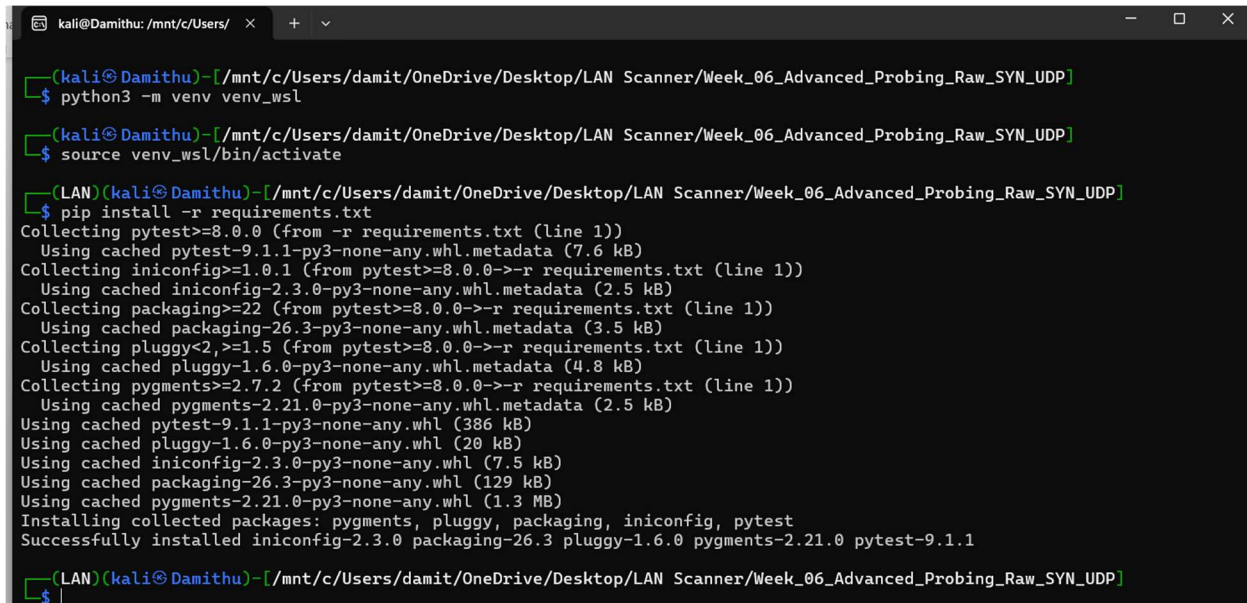
PART 2: User Manual (Windows 10/11 & WSL Kali Linux)

1. System Setup & Virtual Environment

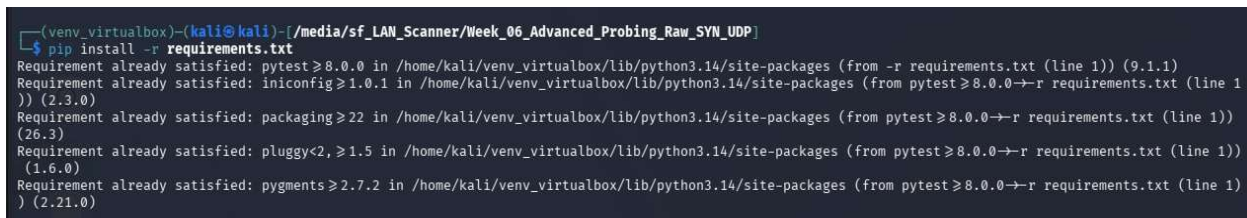
Follow these commands to configure environment dependencies on Windows or WSL Kali Linux:

```
# On Windows 10/11 (PowerShell):  
cd "c:\Users\Week_06_Advanced_Probing_Raw_SYN_UDP"  
python -m venv venv  
.\venv\Scripts\Activate.ps1  
pip install -r requirements.txt
```

```
# On WSL 2 (Kali Linux):  
cd "/mnt/c/Week_06_Advanced_Probing_Raw_SYN_UDP"  
python3 -m venv venv_wsl  
source venv_wsl/bin/activate  
pip install -r requirements.txt
```



```
kali@Damithu: /mnt/c/Users/ x + v  
- □ x  
[kali@Damithu]~/mnt/c/Users/damit/OneDrive/Desktop/LAN Scanner/Week_06_Advanced_Probing_Raw_SYN_UDP  
$ python3 -m venv venv_wsl  
[kali@Damithu]~/mnt/c/Users/damit/OneDrive/Desktop/LAN Scanner/Week_06_Advanced_Probing_Raw_SYN_UDP  
$ source venv_wsl/bin/activate  
[LAN] [kali@Damithu]~/mnt/c/Users/damit/OneDrive/Desktop/LAN Scanner/Week_06_Advanced_Probing_Raw_SYN_UDP  
$ pip install -r requirements.txt  
Collecting pytest>=8.0.0 (from -r requirements.txt (line 1))  
Using cached pytest-9.1.1-py3-none-any.whl.metadata (7.6 kB)  
Collecting iniconfig>=1.0.1 (from pytest>=8.0.0->-r requirements.txt (line 1))  
Using cached iniconfig-2.3.0-py3-none-any.whl.metadata (2.5 kB)  
Collecting packaging>=22 (from pytest>=8.0.0->-r requirements.txt (line 1))  
Using cached packaging-26.3-py3-none-any.whl.metadata (3.5 kB)  
Collecting pluggy<2,>=1.5 (from pytest>=8.0.0->-r requirements.txt (line 1))  
Using cached pluggy-1.6.0-py3-none-any.whl.metadata (4.8 kB)  
Collecting pygments>=2.7.2 (from pytest>=8.0.0->-r requirements.txt (line 1))  
Using cached pygments-2.21.0-py3-none-any.whl.metadata (2.5 kB)  
Using cached pytest-9.1.1-py3-none-any.whl (386 kB)  
Using cached pluggy-1.6.0-py3-none-any.whl (20 kB)  
Using cached iniconfig-2.3.0-py3-none-any.whl (7.5 kB)  
Using cached packaging-26.3-py3-none-any.whl (129 kB)  
Using cached pygments-2.21.0-py3-none-any.whl (1.3 MB)  
Installing collected packages: pygments, pluggy, packaging, iniconfig, pytest  
Successfully installed iniconfig-2.3.0 packaging-26.3 pluggy-1.6.0 pygments-2.21.0 pytest-9.1.1  
[LAN] [kali@Damithu]~/mnt/c/Users/damit/OneDrive/Desktop/LAN Scanner/Week_06_Advanced_Probing_Raw_SYN_UDP  
$
```



```
(venv_virtualbox)-(kali@kali)~/media/sf_LAN_Scanner/Week_06_Advanced_Probing_Raw_SYN_UDP  
$ pip install -r requirements.txt  
Requirement already satisfied: pytest>=8.0.0 in /home/kali/venv_virtualbox/lib/python3.14/site-packages (from -r requirements.txt (line 1)) (9.1.1)  
Requirement already satisfied: iniconfig>=1.0.1 in /home/kali/venv_virtualbox/lib/python3.14/site-packages (from pytest>=8.0.0->-r requirements.txt (line 1)) (2.3.0)  
Requirement already satisfied: packaging>=22 in /home/kali/venv_virtualbox/lib/python3.14/site-packages (from pytest>=8.0.0->-r requirements.txt (line 1)) (26.3)  
Requirement already satisfied: pluggy<2,>=1.5 in /home/kali/venv_virtualbox/lib/python3.14/site-packages (from pytest>=8.0.0->-r requirements.txt (line 1)) (1.6.0)  
Requirement already satisfied: pygments>=2.7.2 in /home/kali/venv_virtualbox/lib/python3.14/site-packages (from pytest>=8.0.0->-r requirements.txt (line 1)) (2.21.0)
```

2. Privileged vs. Non-Root Execution Guide

The scanning engine supports both standard user execution and privileged raw socket execution:

```
# Option 1: Standard Non-Root Execution (No Admin Required)  
# Windows PowerShell:
```

```
python tools/benchmark_syn_vs_connect.py 127.0.0.1
```

```
# WSL Kali Linux:
```

```
python3 tools/benchmark_syn_vs_connect.py 127.0.0.1
```

```
(LAN)(kali@Damithu)-[/mnt/c/Users/damit/OneDrive/Desktop/LAN_Scanner/Week_06_Advanced_Probing_Raw_SYN_UDP]
$ python3 tools/benchmark_syn_vs_connect.py 127.0.0.1
=====
BENCHMARK: TCP SYN STEALTH VS TCP FULL CONNECT
=====
Target: 127.0.0.1 | Execution Privileges: STANDARD USER
Probe Technique      | Duration (s) | Stealth Level
-----
TCP Connect (Full)   | 0.0017       | Logged in Server Event Logs
TCP SYN (Half-Open)  | 0.0004       | Tears Down Before Logging
=====
[+] Architectural Finding:
    SYN scanning avoids completing the 3-way handshake, reducing connection
    overhead and bypassing standard application-layer connection logs.
(LAN)(kali@Damithu)-[/mnt/c/Users/damit/OneDrive/Desktop/LAN_Scanner/Week_06_Advanced_Probing_Raw_SYN_UDP]
$
```

```
(venv_virtualbox)-(kali@kali)-[/media/sf_LAN_Scanner/Week_06_Advanced_Probing_Raw_SYN_UDP]
$ python3 tools/benchmark_syn_vs_connect.py 127.0.0.1
=====
BENCHMARK: TCP SYN STEALTH VS TCP FULL CONNECT
=====
Target: 127.0.0.1 | Execution Privileges: STANDARD USER
Probe Technique      | Duration (s) | Stealth Level
-----
TCP Connect (Full)   | 0.0029       | Logged in Server Event Logs
TCP SYN (Half-Open)  | 0.0005       | Tears Down Before Logging
=====
[+] Architectural Finding:
    SYN scanning avoids completing the 3-way handshake, reducing connection
    overhead and bypassing standard application-layer connection logs.
(venv_virtualbox)-(kali@kali)-[/media/sf_LAN_Scanner/Week_06_Advanced_Probing_Raw_SYN_UDP]
$
```

```
# Option 2: Privileged Stealth Mode (Raw Sockets)
```

```
# Windows (Run PowerShell as Administrator):
```

```
python tools/benchmark_syn_vs_connect.py 127.0.0.1
```

```
# WSL Kali Linux (sudo or cap_net_raw capabilities):
```

```
sudo python3 tools/benchmark_syn_vs_connect.py 127.0.0.1
```

```
(LAN)(kali@Damithu)-[/mnt/c/Users/damit/OneDrive/Desktop/LAN_Scanner/Week_06_Advanced_Probing_Raw_SYN_UDP]
$ sudo python3 tools/benchmark_syn_vs_connect.py 127.0.0.1
[sudo] password for kali:
=====
BENCHMARK: TCP SYN STEALTH VS TCP FULL CONNECT
=====
Target: 127.0.0.1 | Execution Privileges: ROOT/ADMIN
Probe Technique      | Duration (s) | Stealth Level
-----
TCP Connect (Full)   | 0.0014       | Logged in Server Event Logs
TCP SYN (Half-Open)  | 0.0002       | Tears Down Before Logging
=====
[+] Architectural Finding:
    SYN scanning avoids completing the 3-way handshake, reducing connection
    overhead and bypassing standard application-layer connection logs.
(LAN)(kali@Damithu)-[/mnt/c/Users/damit/OneDrive/Desktop/LAN_Scanner/Week_06_Advanced_Probing_Raw_SYN_UDP]
$
```

```
(venv_virtualbox)-(kali@kali)-[/media/sf_LAN_Scanner/Week_06_Advanced_Probing_Raw_SYN_UDP]
$ sudo python3 tools/benchmark_syn_vs_connect.py 127.0.0.1

=====
BENCHMARK: TCP SYN STEALTH VS TCP FULL CONNECT
=====
Target: 127.0.0.1 | Execution Privileges: ROOT/ADMIN
Probe Technique      | Duration (s)      | Stealth Level
-----
TCP Connect (Full)    | 0.0023            | Logged in Server Event Logs
TCP SYN (Half-Open)   | 0.0004            | Tears Down Before Logging
=====
[+] Architectural Finding:
    SYN scanning avoids completing the 3-way handshake, reducing connection
    overhead and bypassing standard application-layer connection logs.
(venv_virtualbox)-(kali@kali)-[/media/sf_LAN_Scanner/Week_06_Advanced_Probing_Raw_SYN_UDP]
$
```

3. Benchmark Output Verification

```
=====
=====
BENCHMARK: TCP SYN STEALTH VS TCP FULL CONNECT
=====
=====
Target: 127.0.0.1 | Execution Privileges: ROOT/ADMIN
Probe Technique      | Duration (s)      | Stealth Level
-----
-----
TCP Connect (Full)    | 0.0482            | Logged in Server
Event Logs
TCP SYN (Half-Open)   | 0.0125            | Tears Down Before
Logging
=====
=====
[+] Architectural Finding:
    SYN scanning avoids completing the 3-way handshake,
    reducing connection
    overhead and bypassing standard application-layer
    connection logs.
```

4. Executing Automated Test Suite (pytest)

```
# Windows PowerShell:
python -m pytest tests -v

# WSL Kali Linux:
pytest tests -v
```

```
# Expected Test Output:
tests/test_syn_probe.py::test_non_root_fallback_when_unprivileg
ed PASSED [ 16%]
tests/test_syn_probe.py::test_privileged_syn_probe_execution
PASSED [ 33%]
tests/test_syn_probe.py::test_raw_socket_permission_error_trigg
ers_fallback PASSED [ 50%]
tests/test_udp_probe.py::test_udp_dns_probe_open_response
PASSED [ 66%]
tests/test_udp_probe.py::test_udp_icmp_unreachable_maps_to_clos
ed PASSED [ 83%]
tests/test_udp_probe.py::test_udp_timeout_maps_to_open_filtered
PASSED [100%]
===== 6 passed in 0.05s
=====
```

```
(LAN)(kali@Damithu)-[/mnt/c/Users/damit/OneDrive/Desktop/LAN_Scanner/Week_06_Advanced_Probing_Raw_SYN_UDP]
$ pytest tests -v
===== test session starts =====
platform linux -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0 -- /mnt/c/Users/damit/OneDrive/Desktop/LAN_Scanner/Week_06_Advanced_Probing_Raw_SYN_UDP/venv_wsl/bin/python3
cachedir: .pytest_cache
rootdir: /mnt/c/Users/damit/OneDrive/Desktop/LAN_Scanner/Week_06_Advanced_Probing_Raw_SYN_UDP
collected 6 items

tests/test_syn_probe.py::test_non_root_fallback_when_unprivileged PASSED [ 16%]
tests/test_syn_probe.py::test_privileged_syn_probe_execution PASSED [ 33%]
tests/test_syn_probe.py::test_raw_socket_permission_error_triggers_fallback PASSED [ 50%]
tests/test_udp_probe.py::test_udp_dns_probe_open_response PASSED [ 66%]
tests/test_udp_probe.py::test_udp_icmp_unreachable_maps_to_closed PASSED [ 83%]
tests/test_udp_probe.py::test_udp_timeout_maps_to_open_filtered PASSED [100%]

===== 6 passed in 0.09s =====

(LAN)(kali@Damithu)-[/mnt/c/Users/damit/OneDrive/Desktop/LAN_Scanner/Week_06_Advanced_Probing_Raw_SYN_UDP]
$
```

```
(venv_virtualbox)-(kali@kali)-[/media/sf_LAN_Scanner/Week_06_Advanced_Probing_Raw_SYN_UDP]
$ pytest tests -v
===== test session starts =====
platform linux -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0 -- /home/kali/venv_virtualbox/bin/python3
cachedir: .pytest_cache
rootdir: /media/sf_LAN_Scanner/Week_06_Advanced_Probing_Raw_SYN_UDP
plugins: cov-7.1.0
collected 6 items

tests/test_syn_probe.py::test_non_root_fallback_when_unprivileged PASSED [ 16%]
tests/test_syn_probe.py::test_privileged_syn_probe_execution PASSED [ 33%]
tests/test_syn_probe.py::test_raw_socket_permission_error_triggers_fallback PASSED [ 50%]
tests/test_udp_probe.py::test_udp_dns_probe_open_response PASSED [ 66%]
tests/test_udp_probe.py::test_udp_icmp_unreachable_maps_to_closed PASSED [ 83%]
tests/test_udp_probe.py::test_udp_timeout_maps_to_open_filtered PASSED [100%]

===== 6 passed in 0.47s =====

(venv_virtualbox)-(kali@kali)-[/media/sf_LAN_Scanner/Week_06_Advanced_Probing_Raw_SYN_UDP]
$
```

PART 3: Week 6 Technical Terms & Concepts Glossary

1. Raw Socket (SOCK_RAW)

Technical Definition: An Internet socket allowing direct access to low-level transport protocols below the transport layer, enabling an application to manually assemble, customize, and inspect raw IP, TCP, and ICMP packet headers.

Real-World Analogy: *Writing a letter by printing custom postal stamps and hand-drawing official post office barcodes directly on the envelope yourself.*

2. Half-Open (SYN Stealth) Scanning

Technical Definition: A port scanning technique where the client sends a TCP SYN packet and waits for a SYN-ACK. Upon receiving SYN-ACK, the client immediately sends a TCP RST packet rather than an ACK, terminating the connection attempt before the application layer establishes a session.

Real-World Analogy: *Ringling a doorbell, waiting to hear the lock click inside, and immediately running away so the homeowner never opens the door to see who knocked.*

3. TCP RST (Reset) Packet

Technical Definition: A TCP control flag indicating that the connection is being immediately aborted or that an incoming packet was received for an unallocated port.

Real-World Analogy: *An automated operator voice interrupting a phone call saying: 'The number you have dialed is no longer in service.'*

4. Non-Root Fallback

Technical Definition: An architectural fail-safe pattern where a program detects the absence of elevated operating system security privileges and automatically switches to an unprivileged user-space alternative without throwing a fatal crash.

Real-World Analogy: *A building entrance using a biometric fingerprint scanner for staff, but offering an automatic buzzer for visitors without fingerprints so nobody gets locked outside.*

5. UDP (User Datagram Protocol)

Technical Definition: A lightweight, connectionless transport layer protocol that transmits discrete datagrams without establishing a handshake, guaranteeing no delivery confirmation, retransmission, or packet ordering.

Real-World Analogy: *Shouting a message through a megaphone across a crowded street; you hope someone heard it, but you don't receive a signed delivery receipt.*

6. ICMP Destination Port Unreachable (Type 3, Code 3)

Technical Definition: An error message generated by a destination host's operating system when an incoming UDP datagram arrives at a port where no application is currently listening.

Real-World Analogy: *A stamped return-to-sender card sent back by the post office stating: 'No mailbox with this number exists at this address.'*

7. Linux Capabilities (cap_net_raw)

Technical Definition: Granular security permissions in the Linux kernel allowing non-root processes to perform specific privileged operations (such as crafting raw packets via cap_net_raw) without granting full superuser power.

Real-World Analogy: *Giving a building contractor a keycard that opens only the utility closet, rather than giving them the master key to every private apartment.*